

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include "commonType.h"
4 #include "dataControl.h"
5 #include "initAndTerm.h"
6
7 LQType *Q;
8 extern int vertexCreatedCnt;
9
10 int insertTotalVertex(int *vCnt) {
11     printf("Input total vertex count\n");
12     while (1) {
13         printf(" ");
14         scanf("%d", vCnt);
15         while (getchar() != '\n');
16         if (*vCnt <= 10 && *vCnt >= 1) {
17             break;
18         }
19         else {
20             printf(" !!!Total vertex must be over 0.\n");
21             continue;
22         }
23     }
24     return *vCnt;
25 }
26
27 int insertEachVertex(graphType *g, int v, int totalCnt) {
28     if ((g->n) + 1 > totalCnt) {
29         printf("\n over vertex\n");
30         return 0;
31     }
32     g->n++;
33     return 0;
34 }
35
36 int requestTargetVertex(int type) {
37     char destVertex;
38     int convVertex = 0;
39
40     if (type == 1)
41         printf("input one vertex to insert near list(ex. a). existed vertex will be overwritten.: ");
42     else
43         printf("input one vertex to insert near list(ex. a).");
44     scanf("%c", &destVertex);
45     while (getchar() != '\n');
46     convVertex = (int)destVertex - 97;
47
48     return convVertex;
49 }
50
51 int inputVertexList(int **vList, int totalCnt, int vCnt) {
52     int i = 0, lNum = 0, convDest = 0;
53     char dest;
54     for (i = 0; i < totalCnt; i++) {
55         (*vList)[i] = -1;
56     }
57     lNum = 0;
58     while (lNum < totalCnt) {
59         printf("input near vertex of \"%c\" [ex] a enter]. When input \"~\" , then terminate input of vertex group. \n", vCnt + 97);
60         printf(" ");
61         scanf("%c", &dest);
62         while (getchar() != '\n');
63         convDest = (int)dest - 97;
64         if (convDest == 29) {
65             if (isNotExistAnyVertex(vList, totalCnt)) {
66                 printf(" !!!Least 1 vertex should be inserted\n");
67                 continue;
68             }
69             else {
70                 break;
71             }
72         }
73         if (isSameDest(vList, totalCnt, convDest)) {
74             printf(" !!!Already input near vertex.\n");
75             continue;
76         }
77         else {
78             (*vList)[lNum] = convDest;
79             lNum++;
80         }
81     }
82     return 0;
83 }
84
85 int isNotExistAnyVertex(int **compVList, int totalCnt) {
86     int i = 0, result = 1;
87     int *tempVList = *compVList;
88
89     for (i = 0; i < totalCnt; i++) {
90         if (tempVList[i] != -1) {
91             result = 0;
92         }
93     }
94     return result;
95 }
96
97 int isSameDest(int **compVList, int totalCnt, int dest) {
98     int i = 0, result = 0;
99
100     for (i = 0; i < totalCnt; i++) {
101         if ((*compVList)[i] == dest) {
102             result = 1;
103         }
104     }
105     return result;
106 }
107
108 int deleteNearList(graphType *g, int groupV) {
109     int i = 0, lNum = 0, convDest = 0;
110     char dest;
111     graphNode *curP = NULL, *nextP = NULL;
112
113     nextP = g->nearListHead[groupV];
114     if (nextP != NULL) {
115         printf("Input one near vertex of \"%c\" to delete [ex] a enter]. \n", groupV + 97);
116         printf(" ");
117         scanf("%c", &dest);
118         while (getchar() != '\n');
119         convDest = (int)dest - 97;
120         if (nextP->vertex == convDest) {
121             curP = nextP;
122             nextP = curP->link;
123             g->nearListHead[groupV] = nextP;
124             free(curP);
125         }
126         while (nextP) {
127             curP = nextP;
128             nextP = curP->link;
129             if (nextP != NULL && nextP->vertex == convDest) {
130                 curP->link = nextP->link;
131                 free(nextP);
132                 nextP = curP->link;
133             }
134         }
135     }
136     else {
137         printf(" !!!list is empty\n");
138     }
139     return 0;
140 }
141
142 int sortNearList(int **vList, int totalCnt) {
143     int *tempSort = (int *)malloc(sizeof(int) * totalCnt);
144     int i, j, min, temp;
145     for (i = 0; i < totalCnt - 1; i++) {
146         min = i;
147         for (j = i + 1; j < totalCnt; j++) {
148             if ((*vList)[j] < (*vList)[min])
149                 min = j;
150         }
151         temp = (*vList)[i];
152         (*vList)[i] = (*vList)[min];
153         (*vList)[min] = temp;
154     }
155     for (i = 0; i < totalCnt; i++) {
156         tempSort[i] = (*vList)[i];
157     }
158     for (i = 0; i < totalCnt; i++) {
159         (*vList)[i] = tempSort[totalCnt - (i + 1)];
160     }
161     for (i = 0; i < totalCnt; i++) {
162         printf("%d(%c) ", (*vList)[i], (*vList)[i] + 97);
163     }
164     printf("\n");
165     free(tempSort);
166     return 0;
167 }
168
169 int eraseNearListOfVertex(graphType *g, int groupV) {
170     graphNode *curP = NULL, *nextP = NULL;
171     nextP = g->nearListHead[groupV];
172     while (nextP) {
173         curP = nextP;
174         nextP = curP->link;
175         free(curP);
176     }
177     g->nearListHead[groupV] = NULL;
178     return 0;
179 }
180
181 int insertEachEdge(graphType *g, int groupV, int v) {
182     graphNode *node;
183     if (groupV >= g->n || v > g->n) {
184         printf("\n no match vertex\n");
185         return 0;
186     }
187     node = (graphNode *)malloc(sizeof(graphNode));
188     node->vertex = v;
189     node->link = g->nearListHead[groupV];
190     g->nearListHead[groupV] = node;
191     return 0;
192 }
193
194 int checkExistAcrossEdge(graphType *g, int groupV, int totalVertex) {
195     int targetV = groupV;
196     int exist = 0;
197     graphNode *p = g->nearListHead[groupV];
198     graphNode *tempP;
199
200     while (p) {
201         tempP = g->nearListHead[p->vertex];
202         exist = vertexAcrossEdge(&tempP, groupV);
203         if (exist != 1) {
204             printf(" !! check vertex %c. There is not exist %c \n", p->vertex + 97, groupV + 97);
205         }
206         p = p->link;
207     }
208     return 0;
209 }
210
211 int vertexAcrossEdge(graphNode **tempV, int groupVnum) {
212     int existResult = 0;
213     graphNode *tempP = *tempV;
214     while (tempP) {
215         if (tempP->vertex == groupVnum) {
216             existResult = 1;
217         }
218         tempP = tempP->link;
219     }
220     return existResult;
221 }
222
223 int printNearList(graphType *g) {
224     int i;
225     graphNode *p;
226
227     printf("\n");
228     for (i = 0; i < g->n; i++) {
229         printf(" vertex list of %c ", i + 97);
230         p = g->nearListHead[i];
231         while (p) {
232             printf(" -> %c", p->vertex + 97);
233             p = p->link;
234         }
235         printf("\n");
236     }
237     return 0;
238 }
239
240 int isEmptyQueue(LQType *LQ) {
241     if (LQ->front == NULL) {
242         printf("\n !!!Queue empty.\n");
243         return 1;
244     }
245     else {
246         return 0;
247     }
248 }
249
250 int enqueue(LQType *LQ, int v) {
251     queueNode *newNode = (queueNode *)malloc(sizeof(queueNode));
252     newNode->n = v;
253     newNode->link = NULL;
254     if (LQ->front == NULL) {
255         LQ->front = newNode;
256         LQ->rear = newNode;
257     }
258     else {
259         LQ->rear->link = newNode;
260         LQ->rear = newNode;
261     }
262     return 0;
263 }
264
265 int dequeue(LQType *LQ) {
266     queueNode *oldNode = LQ->front;
267     int v;
268     if (isEmptyQueue(LQ)) {
269         return 0;
270     }
271     else {
272         v = oldNode->n;
273         LQ->front = LQ->front->link;
274         if (LQ->front == NULL) {
275             LQ->rear = NULL;
276         }
277         free(oldNode);
278         return v;
279     }
280 }
281
282 int bfsNearList(graphType *g, int v) {
283     graphNode *GN;
284     int i = 0;
285
286     Q = createLQ();
287     g->visited[v] = TRUE;
288     printf("\n%c ", v + 65);
289     enqueue(Q, v);
290     while (!isEmptyQueue(Q)) {
291         v = dequeue(Q);
292         for (GN = g->nearListHead[v]; GN; GN = GN->link) {
293             if (!g->visited[GN->vertex]) {
294                 g->visited[GN->vertex] = TRUE;
295                 printf("%c ", GN->vertex + 65);
296                 enqueue(Q, GN->vertex);
297             }
298         }
299     }
300 }
301
302 for (i = 0; i < vertexCreatedCnt; i++) {
303     g->visited[i] = FALSE;
304 }
305
306 releaseLQ(&Q);
307 return 0;
308 }
```